sw-vague-smoke

An AgentMPI run analysis

Abstract

This run does for the vague-specification regime what `sw-smoke` did for the precise one: four ranks under the deterministic surrogate executor take two `minidb` modules each through one round of the pipeline — broadcast, publish interfaces into the RMA window, fence, read the fourteen dependency slots, implement, barrier, gather, oracle, report, blame, agree — in 0.146 s, with all six checkable collectives matching their closed forms and no contract violated. Its value is its position in the campaign: recorded at 08:07:36, an hour after `sw-smoke` and 53 minutes before the first real vague-specification run, it is the evidence that `-vague-spec` breaks nothing downstream of the substitution. "Nothing" is meant literally. The two smokes have the same 128 events, the same event vocabulary, the same six collectives with the same algorithms and 20 messages, the same 16 agent calls and the same 496 completion tokens; the only trace-visible differences are token volumes, and they reconcile to the 15 tokens by which the vague module table is longer than the one it replaces. Under a deterministic executor that identity is the point rather than a curiosity — it shows control flow does not depend on specification content — but it is also why neither run measures anything. The executor is `function`, the tree produced is 24 lines of stubs scoring 0/60 by construction, and the $0.095 in the headline table prices prompts no model read.

1 What this run is

property	value
run	`sw-vague-smoke`
experiment	`software`
campaign label	`sw-vague-smoke`
declared world size	4
ranks appearing in the log	4
executors	`function` × 4
events	128
wall time	0.15 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 4 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	64.5%	rank-seconds blocked in collectives, of those available
coordination in flight	82.1%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	16	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	20	17 eager, 3 rendezvous
tokens sent	5,717	189 deferred under rendezvous
tokens in / out	29,193 / 496	prompt and completion
cost	$0.0950	0.1916 $/kTok out

3 What the analysis notices

- [note] Collectives account for 64.5% of the run's rank-seconds, so more of the pool's time went to coordination than to work.
- [ok] All 6 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

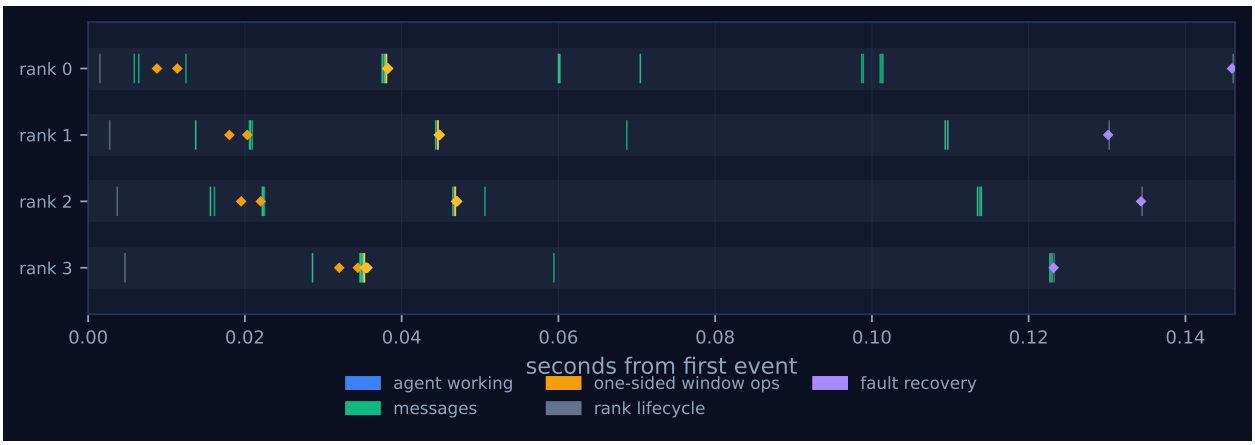

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

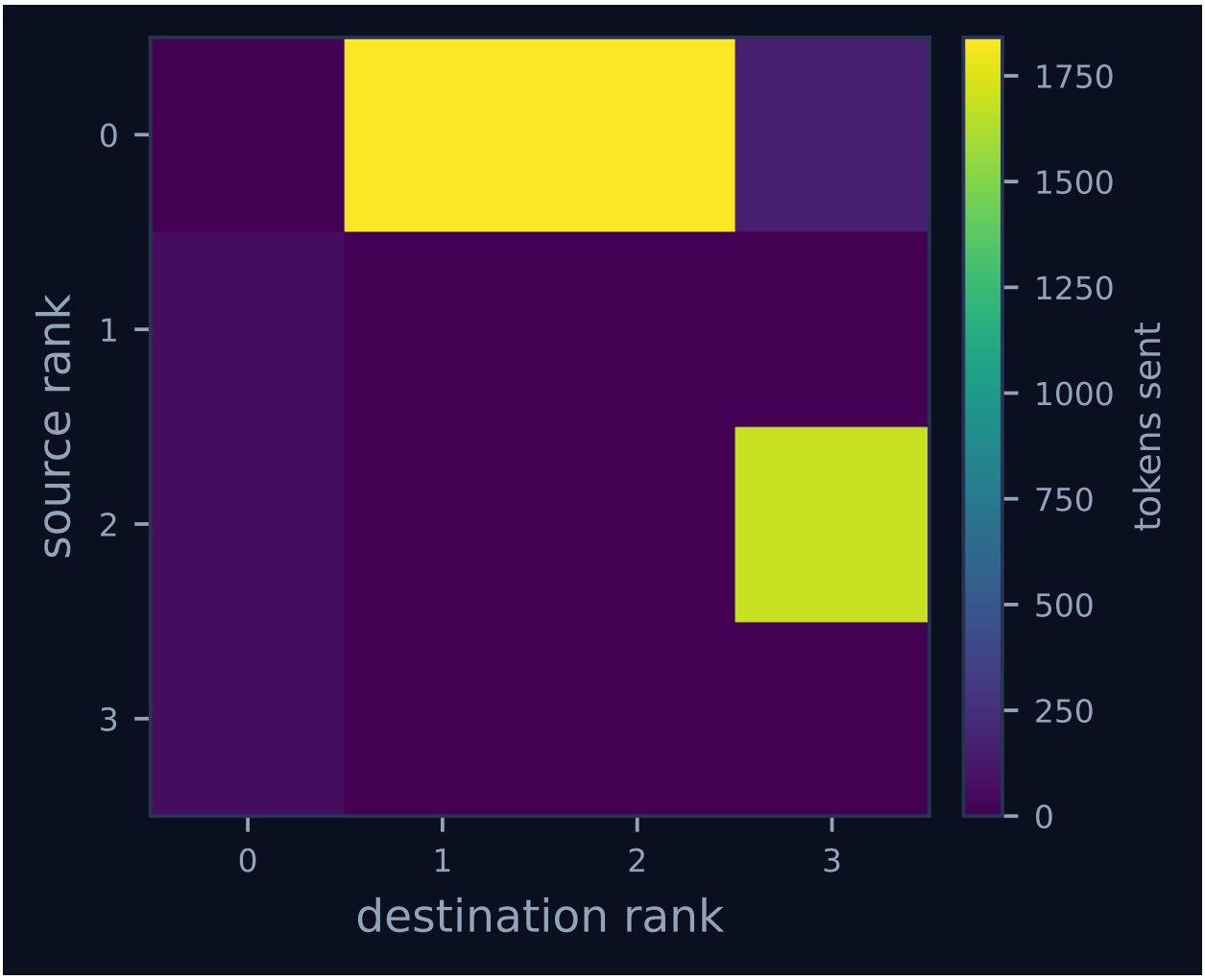

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

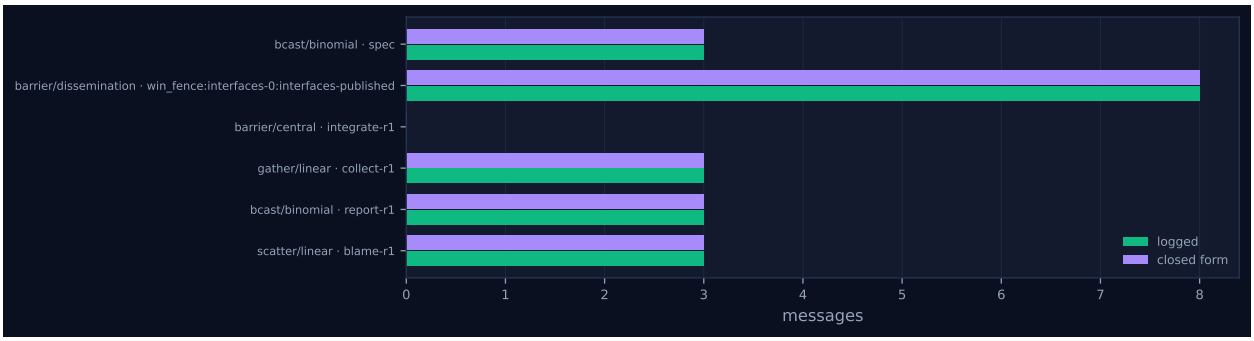

Figure 3: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	4	9	5	3,834	0.0	finalized
1	0.00	0.0	0	4	3	5	65	0.0	finalized
2	0.00	0.0	0	4	5	5	1,753	0.0	finalized
3	0.00	0.0	0	4	3	5	65	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)
bcast	binomial	spec	4	4	2	2	3	3	4,623	0.02
barrier	dissemination	win_fence:interfaces-0:interfaces-published	4	4	0	2	8	8	8	0.03
barrier	central	integrate-r1	4	4	0	2	0	0	0	0.02
gather	linear	collect-r1	4	4	1	3	3	3	189	0.01
bcast	binomial	report-r1	4	4	2	2	3	3	441	0.06
scatter	linear	blame-r1	4	4	1	3	3	3	456	0.00

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 contains no work bars, and that is the first thing to account for. With `-executor function` every one of the sixteen `agent.call` events carries `latency_s: 0.0`, so the calls are instants and the four lanes hold only message ticks, amber window diamonds and four violet `ft.agree` marks near the right edge, across 0.146 s. Every occupancy figure in the headline table follows from that and none describes scheduling: achieved parallelism 0.00×, parallel efficiency 0.0 %, load imbalance 0.00×, idle fraction 100.0 %. The [note] that collectives take 64.5 % of rank-seconds is arithmetically correct and interpretively empty, because the denominator contains no work.

The phase sequence is what remains, and it is exactly the harness's. Four `rank.init`s and the `spec` broadcast open the run; eight `iface` calls each followed immediately by a `win.put` carry it to $t = 0.035$ s; the `interfaces-published` fence releases each lane in turn, and each then issues a `win.sync` and its own dependency reads — five for rank 3, which owns `functions` and `api`, three for rank 0, which owns `errors` and `parser`; eight `impl:*:r1` calls follow, then the `integrate-r1` barrier and the gather at $t \approx 0.07$ s, the report broadcast and blame scatter at $t \approx 0.10$–0.12 s, and agreement and finalisation. Watching that complete without a stall, a timeout or a contract rejection is the whole purpose of the run. Figure 2 is a two-level binomial tree over a nearly empty plane — rank 0 sends 1,841 tokens to each of ranks 1 and 2 in four messages apiece, rank 2 forwards

1,689 to rank 3, and the fan-in is 63 to 64 tokens per rank — which is `sw-smoke`'s matrix with every cell about 1 % larger, for the reason given below.

7 Interpretation

The configuration is `-ranks 4 -rounds 1 -vague-spec -executor function`, with shared interfaces, locks and review nominally enabled and smoke-scale guards (`barrier_timeout 20 s`, `job_timeout 60 s`). Module assignment is $i \bmod p$: rank 0 owns `errors` and `parser`, rank 1 `tokens` and `planner`, rank 2 `nodes` and `engine`, rank 3 `functions` and `api`. The specific thing under test is `apply_vague_layout`, which locates the specification's "## Module layout" heading, replaces everything up to "## Examples" with a table naming each module's responsibility and permitted imports but no signatures, and raises if either marker is missing. The substitution is not itself visible in the event log; what the log shows is that it happened — the specification broadcast carries 1,541 tokens rather than 1,526 — and that nothing after it noticed. Within that scope the run is clean: six collective invocations, all complete, all matching their predicted message counts, twenty messages, and zero contract violations over sixteen contract-checked calls.

`-rounds 1` bounds the coverage, and the bound bites harder here than in the precise smoke. Both the cross-review phase and the interface renegotiation are guarded by `round_no < cfg.rounds`, false in a single-round run, so neither executes: no `topo.graph_create`, no `coll.neighbor_allgather`, no `win.lock` or `win.unlock`, and all eight slots ending at version 1. The vague layout exists precisely to make the window load-bearing — with signatures withheld, a published interface is the only way a rank can learn what its dependencies expose — and the mechanism for correcting a bad interface, the locked renegotiation, is exactly the half this run does not reach.

The comparison with `sw-smoke` is the run's main content. The pair differs in one configuration flag and the traces are the same trace: 128 events each, identical `kind_counts` across all sixteen event kinds, identical `role_counts`, the same collectives in the same order with the same algorithms, the same 20 messages and 16 agent calls, identical completion token totals. That is worth a sentence rather than a shrug. Under a deterministic executor `synthetic_agent` keys its reply on the call label alone and ignores the prompt entirely, so no branch in the harness can depend on what the specification says; identical event counts are therefore the expected outcome, and confirming them is a modest but real check, since a vague substitution that changed the number of window operations or collectives would mean something in the harness was reading the specification where it should not. What the pair cannot show is any consequence of vagueness, the only component that could respond to it having been replaced by a lookup table.

Every difference is token volume, and it reconciles.

quantity	`sw-smoke`	`sw-vague-smoke`	Δ
specification tokens	1,526	1,541	+15
`spec` broadcast tokens	4,578	4,623	+45
prompt tokens	28,965	29,193	+228
tokens sent	5,642	5,717	+75
cost (USD)	0.0943	0.0950	+0.0007
wall (s)	0.119	0.146	+0.027

`VAGUE_LAYOUT` makes the specification 54 characters and 15 tokens longer. It is embedded in all

sixteen prompts, which accounts for the 228-token prompt difference at 14.25 tokens per prompt, and broadcast to three ranks, which accounts for the 45-token collective difference. That 0.79 % prompt increase is the entire measurable cost of the vague regime under this executor; with real agents the cost is in what agents do with the missing signatures, which is what `vague-sw-p8-vague-shared` and `vague-sw-p8-vague-noshared` measure. The 0.027 s wall difference is scheduling noise on a sub-second run and should not be read as anything. One residual difference is not attributable to the specification: this run's oracle reports `n_total: 60` and a traceback at line 181 of `acceptance.py`, against 59 and line 174 for `sw-smoke`. The suite gained a case in the 65 minutes between them, so the pair differs in oracle version as well as regime, and that difference propagates into the token columns through the report and blame payloads (147 and 152 tokens per message here, 142 and 147 there).

Incidental coverage worth recording: `algorithms.gather` chooses `"linear" if p <= 4 else "binomial"` and `scatter` follows the same rule, so this run and `sw-smoke` are the only software runs on the linear branch; `sw-rev-smoke` at $p = 6$ and all four real-agent runs at $p = 8$ take the binomial one. The collectives table shows `rounds` 1 for both linear invocations against a prediction of 3 while the message counts agree at 3, which is not a defect: a linear gather issues all $p - 1$ transfers without waiting and counts one round, whereas the closed form counts $p - 1$ because they serialise at the root.

8 Threats to this reading

The governing threat is that none of this is a measurement. `executors: function` × `4`: nothing here is evidence about agent behaviour, model latency, code quality, or whether a vague specification is harder to work from. The 0.146 s wall time is SQLite writes and event logging, the sixteen agent calls are canned dictionaries, and the $0.0950 and 29,193 prompt tokens price prompts that were assembled and counted but never sent; the 0.19157 $/kTok-out figure in `metrics.json` is that prompt bill divided by 496 stub completion tokens.

The zero acceptance score is a construction, not a failure. The oracle reports 0 of 60 with `ImportError: cannot import name 'query' from 'minidb.api'` because `synthetic_agent` returns `def stub_api(): return None`. That is the intended behaviour of the surrogate and the reason the blame scatter and the agreement carry non-trivial payloads; the subsequent `agree(false)` is not a negative result.

The pair is also not controlled. `sw-smoke` and this run differ in specification regime, in harness commit (`b0b652d` against `7a3331a`) and in acceptance-suite version. The comparison above survives because the surrogate makes control flow independent of all three, but it should not be extended to any quantity that depends on the code that changed between those commits.

Two smaller cautions. The four `ft.agree` events are counted under the recovery role, so `role_counts` reports 4 recovery events against 0 trouble events and the dashboard reads "fault recovery 4" beside FAILURES 0; nothing was recovered from, the agreement recording `voters: [0, 1, 2, 3]` and `excluded: []`. And $p = 4$ divides the module count evenly, so uneven ownership — which `sw-rev-smoke` does produce at $p = 6$ — never arises, and the surrogate's zero-cost calls would conceal its effect even if it did.